



Ingeniería de Videojuegos: Fundamentos y aplicaciones interactivas.

Game Design Document

Team Core Gaming

Profesor: Joaquín Macías.

Estudiantes : Rodrigo Pereira

Patricio Neira

Joaquin Pozo.

24 de Julio de 2026

1. Identificación

Nombre del proyecto:	Unknown Planet
Equipo:	Team Core Gaming
Integrantes:	Rodrigo Pereira, Patricio Neira, Joaquín Pozo
Motor:	Godot Engine 4.6 (GDScript)
Plataforma:	Windows (exportación nativa .exe)
Género:	Farming simulator + supervivencia (top-down 2D)
Audiencia:	12 años en adelante; jugadores casuales al inicio, con curva hacia experiencia media en niveles avanzados

Unknown Planet es un videojuego de supervivencia y exploración en 2D donde un astronauta debe aterrizar forzosamente en un planeta hostil, abrirse paso por distintos biomas (Planeta Desconocido, Catacumbas y Tormenta de Hielo), derrotar enemigos (Slimes), recolectar recursos y sobrevivir hasta encontrar la salida de cada nivel para escapar.

2. Visión y propuesta de valor

2.1. Elevator Pitch

Un astronauta cae forzosamente en un planeta hostil y debe sobrevivir farmando recursos, combatiendo enemigos y reparando su nave para escapar. El ciclo de juego alterna entre una fase de preparación y una fase de exploración bajo presión, donde slimes autónomos patrullan y persiguen al jugador por tres niveles cada vez más hostiles. La tensión crece progresivamente mediante ambientes que cambian las reglas del juego.

2.2. Diferenciador

A diferencia de perseguir en línea recta, los enemigos navegan realmente el escenario mediante NavigationAgent2D, y cada nivel modifica las reglas de movimiento y visibilidad del jugador (ventisca, laberinto), obligando a adaptar la estrategia en cada nivel.

3. Diseño del juego

Capa	Contenido actual del proyecto
Mecánicas	Movimiento en 4 direcciones
	Ataque melee
	Completar niveles encontrando la salida
	Farmear recursos
Dinámicas esperadas	El jugador decide si esquivar slimes o enfrentarlos antes de avanzar
	El jugador explora el mapa buscando la salida y recursos minables
Estéticas objetivo	Desafío: los slimes presionan al jugador y el ambiente cambia por nivel (oscuridad, tormenta)

Tabla 1: MDA

La mecánica central es el ataque melee del personaje, ya que el juego exige sobrevivir a asaltos de enemigos constantemente, siendo esta la herramienta fundamental del gameplay. Al ejecutarla, el jugador siente que golpea a un enemigo con una espada: si la hitbox de ataque (Area2D) conecta con el enemigo dentro de su rango, se recibe feedback audiovisual inmediato (animación + sonido) y se inflige daño; si falla, el golpe solo se mueve en el aire, sin consecuencias.

3.1 Layout de Niveles

3.1.1. Nivel 1: Introducción (Planeta Desconocido): pocos enemigos, sirve como introducción al movimiento y las mecánicas básicas. La guía visual implícita se marca con tiles de camino más amigables.

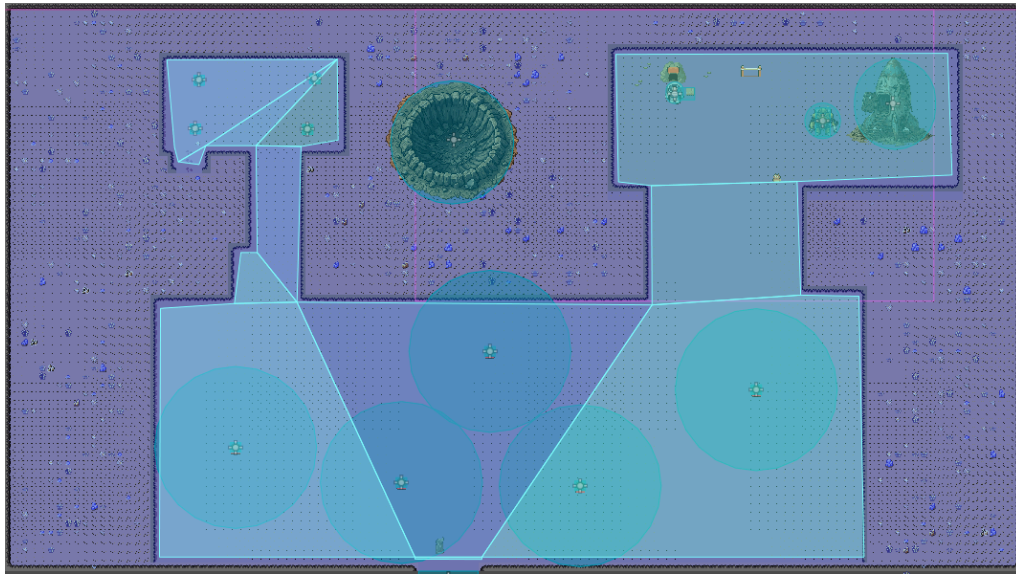


Figura 1: Layout nivel 1

3.1.2. Nivel 2: Laberinto (Catacumbas): acota el rango de visión del jugador y enseña que los niveles pueden cambiar drásticamente. La guía visual son marcas de piedras dispuestas en patrones de a tres, que indican el camino correcto del laberinto.

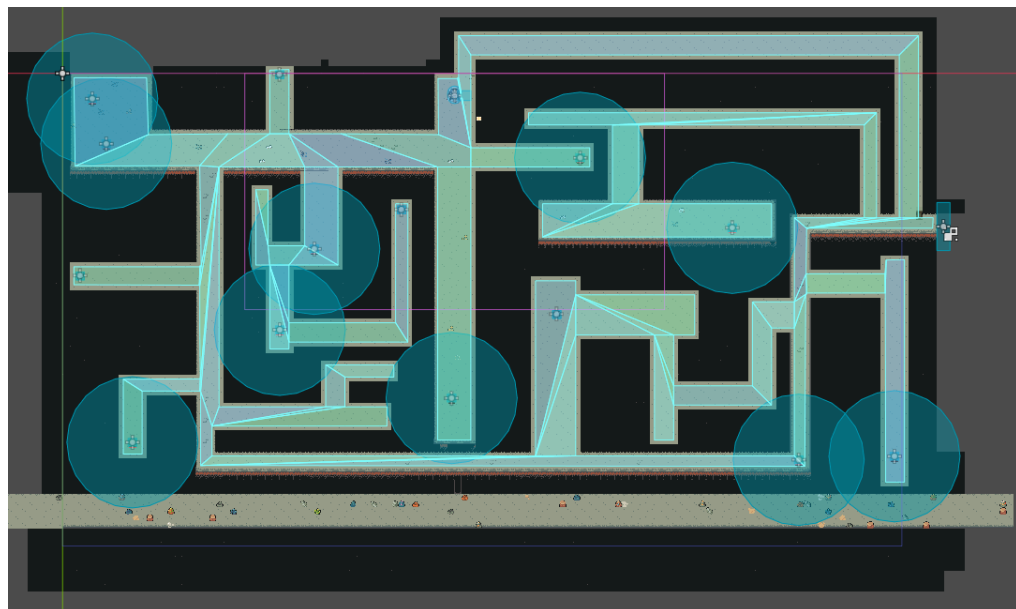


Figura 2: Layout nivel 2

3.1.3. Nivel 3: Ventisca (Tormenta de Hielo): tiene la máxima dificultad, más enemigos, ventisca que reduce la visibilidad y hace que el personaje se deslice mucho más que los enemigos. La guía visual son postes que marcan el camino hacia la salida, reforzados con calaveras dispuestas como narrativa ambiental (otros exploradores que no lo lograron), terminando en el bloque de tierra donde están los últimos minerales y la salida (retorno al nivel 1).

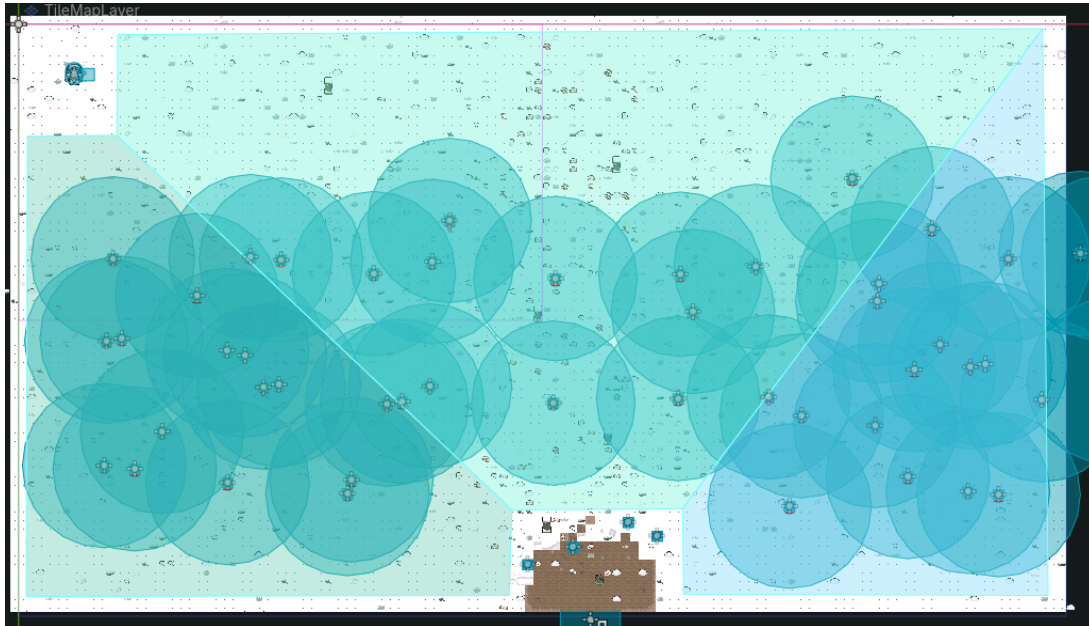


Figura 3: Layout nivel 3

3.2. Descripción del agente y sus 3 estados

Agente: Slime (enemigo hostil), presente en los tres niveles. Estructura de control: FSM (Finite State Machine), elegida sobre un Behavior Tree porque el agente maneja solo 3 comportamientos estables (Patrol, Chase, Attack), sin necesidad de sub-árboles reutilizables.

Estado	Comportamiento	Transición
PATROL	Patrulla el nivel de forma autónoma	→ CHASE si el jugador entra al rango de detección (Area2D "Sight")
CHASE	Persigue al jugador usando NavigationAgent2D sobre un NavMesh dibujado a mano	→ PATROL si el jugador sale del rango · → ATTACK si la distancia $\leq$ attack_range
ATTACK	Ataca cuerpo a cuerpo, inflige daño con cooldown	→ CHASE si el jugador se aleja (con histéresis de attack_range $\times$ 1.3 para evitar oscilación)

Tabla 2: Detalle slime

Cada estado implementa enter_state() / exit_state() y una función de actualización continua controlada mediante un match sobre un enum State en _physics_process().

4. Arquitectura técnica

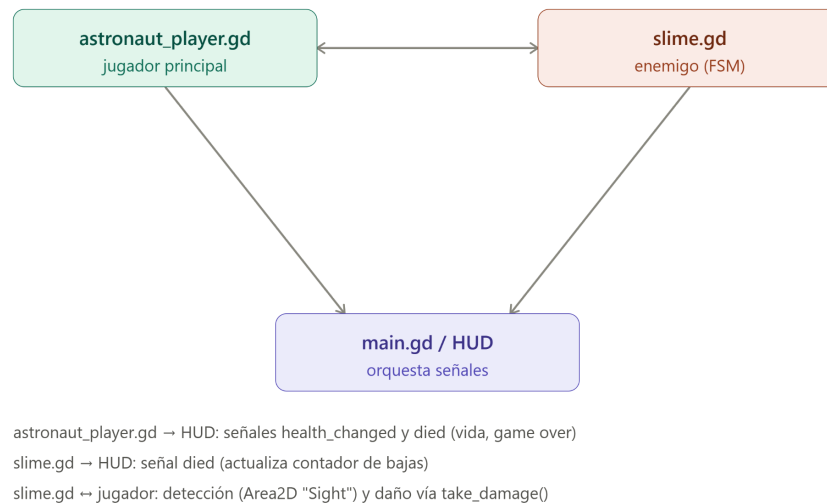


Figura 4: Resumen arquitectura componentes principales

4.1. Componentes principales

4.1.1. astronaut_player.gd:

Controla input, movimiento con delta time (aceleración/fricción), StateMachine de movimiento y acción, ataque, farmeo, interacción y sistema de vida (señal health_changed).

4.1.2. slime.gd:

FSM de 3 estados, detección por Area2D de rango, navegación vía NavigationAgent2D, sistema de daño/knockback y señal died.

4.1.3. main.gd:

Orquesta las señales del juego: health_changed actualiza corazones, died (del jugador) dispara game over, died (de cada slime) actualiza el contador de bajas.

4.2. Decisiones técnicas relevantes

4.2.1. GDScript sobre C#: se priorizó la agilidad de desarrollo y la sintaxis más simple, dado que el rendimiento bruto de C# no era necesario para la escala del proyecto.

4.2.2. Sable melee en lugar de arma a distancia: una pistola láser requería física de proyectiles (instanciación, trayectorias, colisiones en movimiento), complejidad desproporcionada para el tiempo disponible. El ataque melee se resuelve con un área de colisión simple activada durante la animación de golpe.

4.2.3. FSM en vez de Behavior Tree para el enemigo: justificado por el bajo número de estados necesarios

4.2.4. NavigationPolygon dibujado a mano: por incompatibilidad del bake automático con TileMapLayer.

4.2.5. Corrección de knockback: se detectó que el knockback del slime (distancia fija en píxeles) podía empujarlo fuera del NavMesh o a través de paredes en el nivel 2. Se corrigió recortando el destino del knockback con un `PhysicsRayQueryParameters2D` contra la capa de colisión del entorno, y ajustando el punto final con `NavigationServer2D.map_get_closest_point()`.

4.3. Deudas técnicas:

4.3.1. Farming: tiene un bug que ocurre de manera random donde el personaje realiza la animación pero no se ejecuta la acción.

4.3.2. Slime Collision: si el personaje colisiona con el slime en la parte superior, el slime queda pegado al personaje y tiene que colisionar con otro objeto o salir del área para despegarse.

5. Proceso de desarrollo

5.1. Tabulación desarrollo

LAB	Contenido	Estado
LAB 1	Propuesta de proyecto	Completado
LAB 2	Prototipo de escena base (jerarquía, delta time)	Completado
LAB 3	Mecánica principal (ataque melee) implementada	Completado
LAB 4	Jugabilidad y diseño de niveles (3 niveles, HUD, sistema de vida)	Completado
LAB 5	IA y sistema de enemigos (FSM del Slime, navegación)	Completado
LAB 6 CIERRE	Prototipo funcional + GDD + presentación	En curso (este documento)

Tabla 2: Proceso de desarrollo

5.2. Resumen de la sesión de playtesting (Bloque B):

Se realizó una sesión con un participante externo de 23 años, con experiencia previa en videojuegos shooter. Los principales puntos de confusión fueron la falta de indicaciones iniciales y la baja visibilidad en el laberinto del nivel 2. A partir de esto se implementaron dos ajustes: un tutorial breve al inicio de la partida que explica el objetivo del juego, y una transición desde la salida del nivel 3 de vuelta al nivel 1, permitiendo al jugador recolectar recursos que le faltaron. El detalle completo de la sesión está documentado en el Documento de Cierre, Parte 2.

6. Dimensión aplicada

6.1. Clasificación del proyecto + justificación.

Unknown Planet es un videojuego con potencial aplicado debido a que su propósito principal es el entretenimiento. Sin embargo, el videojuego contempla ciertos desafíos para el jugador en la experiencia de juego, tales como niveles con dificultad incremental; exploración y recolección de recursos en ambientes adversos; combate cuerpo a cuerpo con múltiples enemigos y supervivencia en base a gestión de recursos limitados.

6.2. Necesidad o valor identificado.

El proyecto es útil para jugadores casuales de 12 años en adelante que buscan una experiencia de supervivencia y farreo con una curva de dificultad progresiva, similar a referentes como Stardew Valley o Minecraft. Más allá del entretenimiento, vale la pena porque combina exploración, gestión de recursos y un combate que obliga al jugador a

planificar su estrategia en lugar de reaccionar únicamente por reflejos.

6.3. Un objetivo de impacto.

Que el jugador sea capaz de reconocer cuándo priorizar la exploración cautelosa sobre la confrontación directa con los enemigos, y también de planificar rutas considerando tanto el entorno como el comportamiento autónomo de los slimes.

6.4. Análisis TAD resumido.

Autonomía: el jugador elige libremente cuándo farmear, combatir o explorar.

Competencia: el sistema de niveles progresivos permite dominar mecánicas antes de enfrentar mayor dificultad.

Relación: no aplica de forma directa (juego single-player).

7. Resultados y conclusiones

7.1. Estado final del prototipo

El prototipo es jugable de principio a fin: los tres niveles están implementados con su mecánica principal (ataque melee), HUD funcional (vida, contador de enemigos), sistema de condición de fin (game over / misión completada) y agente enemigo autónomo con FSM de 3 estados y navegación real por el escenario.

7.2. Aprendizajes del equipo:

1. El aprendizaje de cómo se desarrolla un videojuego desde cero, incluyendo el diseño, el desarrollo del código, la implementación de las mecánicas desde un punto de vista conceptual a uno mecánico e informático.
2. La importancia de contar con todos los assets, puesto que unos recursos visuales y musicales bien definidos desde un comienzo facilitan posteriormente el desarrollo de la lógica de programación.
3. La dificultad de terminar un videojuego. El proceso de creación y desarrollo de la idea no se nos complicó tanto como la integración de todos los aspectos del proyecto. Es fácil hacer una mecánica de ataque en solitario, pero al tener que implementarla con el ambiente (ya sean los enemigos y el espacio donde se pueden mover los personajes) tomó mucho más tiempo que el desarrollo básico de cada parte por separado, la integración y el pulir esto fue lo más complicado ya que al desarrollar cosas en diferentes momentos y por diferentes personas, el pulido final se tuvo que realizar con mucho cuidado.

7.3. Tres cosas que hubiéramos hecho de diferente manera:

1. La creación de los assets. Hubiéramos preferido tener todos los componentes de arte para el desarrollo del juego desde un principio, ya que al obtener assets de diferentes fuentes, se perdió tiempo en la integración de los diferentes estilos y tamaños.
2. La planeación inicial del desarrollo. Al no tener competencias de desarrollo de videojuegos, nuestra planeación inicial fue más de desarrollo de software que de videojuegos. Por lo tanto, muchos puntos y aspectos que considerábamos importantes, no lo fueron y tuvimos que adaptarnos.
3. Aprender los fundamentos del motor de desarrollo. Nos hubiera gustado aprender el motor desde un punto de vista teórico, ya que recurrimos a técnicas o conceptos de alto nivel que no comprendimos bien en un principio. Esto nos hizo volver atrás a cambiar cosas que ya teníamos listas para mejorarlas con el conocimiento adquirido.



8. Referencias

1. Assets Godot: <https://godotengine.org/asset-library/asset>
2. Assets cute fantasy rpg (objetos): <https://kenmi-art.itch.io/cute-fantasy-rpg>
3. Assets ninja adventure (escenarios): <https://pixel-boy.itch.io/ninja-adventure-asset-pack>
4. Documentación Godot: [:https://docs.godotengine.org/en/stable/](https://docs.godotengine.org/en/stable/)
5. Freesound (efecto de sonido del ataque):
<https://freesound.org/people/joe93barlow/sounds/78685/>
6. Material del curso: Ingeniería de Videojuegos: Fundamentos y Aplicaciones Interactivas, Departamento de Ingeniería Informática, USACH.
7. Repositorio del proyecto: https://github.com/rodrigo-alonso-pereira/unknown_planet_ingevi
8. Recurso comunitario destacado GDQuest: <https://www.gdquest.com/tutorial/#gamedev>
9. Tu primer juego (Godot):
https://docs.godotengine.org/en/3.0/getting_started/step_by_step/your_first_game.html